

Tarefa 12: Desempenho vs. Escalabilidade

Avaliação de escalabilidade de Navier-Stokes com OpenMP no supercomputador do NPAD

Professor: Samuel Xavier de Souza
Aluno: Vinícius Silva do Carmo
Setembro de 2026

1. Objetivo

Avaliar a escalabilidade do código de Navier-Stokes em um nó de computação do NPAD, identificar os gargalos de escalabilidade e reportar o progresso em versões sucessivas do código otimizado, comentando a escalabilidade, a escalabilidade fraca e a escalabilidade forte de cada versão.

O núcleo numérico é o da Tarefa 11 — difusão viscosa $\partial u / \partial t = \nu \nabla^2 u$, resolvida por um stencil de 5 pontos com Euler explícito e $r = \nu \cdot \Delta t / \Delta x^2 = 0,01 \ll 0,25$ —, agora com a grade dimensionada em tempo de execução, pré-requisito para medir escalabilidade fraca.

2. Ambiente de Execução

Item	Valor
Máquina	Supercomputador NPAD/UFRN, partição amd-512, nó exclusivo
Processador	2 × AMD EPYC 7713 (Milan), 64 núcleos por socket
Núcleos físicos	128 (SMT desativado via <code>--hint=compute_bound</code>)
Domínios NUMA	8 (16 núcleos cada)
Memória	512 GB DDR4-3200, 8 canais por socket (~409 GB/s de pico)
Cache L3	32 MB por CCD; 512 MB agregados no nó
Compilador	GCC 14.2.0, <code>-O3 -march=native -fopenmp</code>
Afinidade	<code>OMP_PLACES=cores, OMP_PROC_BIND=spread</code>

A reserva exclusiva do nó não é detalhe de conforto: medir escalabilidade em nó compartilhado mistura o comportamento do código com a disputa de banda de memória de outros jobs — e a banda é justamente a grandeza sob investigação.

3. Versões do Código

Cinco versões do mesmo cálculo, cada uma removendo um gargalo da anterior. Todas estão em um único arquivo, selecionadas por argumento de linha de comando.

Versão	O que muda	Gargalo removido
v0	Referência sequencial de 1 thread	—
v1	<code>omp parallel for</code> + <code>memcpy</code> do grid a cada passo	— (versão ingênua)
v2	Troca de ponteiros no lugar da cópia	Trecho serial (Amdahl)
v3	Inicialização paralela com a mesma partição do cálculo	Localidade NUMA
v4	Um só <code>omp parallel</code> em torno do laço do tempo	Fork/join por passo

4. Validação

Otimização que muda o resultado não é otimização. As cinco versões foram executadas sobre a mesma condição inicial e o campo final comparado por checksum, com 128 threads:

Versão	Checksum Σu	Pico	Resultado
v0-sequencial	1,570796326795e+03	8,630114	(referência)
v1-memcpy	1,570796326795e+03	8,630114	idêntico
v2-doublebuf	1,570796326795e+03	8,630114	idêntico
v3-firsttouch	1,570796326795e+03	8,630114	idêntico
v4-regiao-unica	1,570796326795e+03	8,630114	idêntico

Os checksums coincidem **bit a bit**, o que era esperado: o stencil é determinístico por célula e não há redução em ponto flutuante, então a ordem das threads não altera o resultado. A massa medida (1,570796e+03) coincide com a integral analítica da gaussiana $2\pi\sigma^2 A$, confirmando que a física permanece correta com 128 threads.

5. Escalabilidade Forte

Grade fixa de 4096×4096 (134 MB por grid), 100 passos, mediana de 3 execuções. Referência sequencial: **1,171 s**.

Threads	v1 (speedup)	v2	v3	v4	Ideal
1	0,6×	1,0×	1,0×	1,0×	1×
2	0,7×	1,7×	2,1×	2,2×	2×
4	0,7×	3,0×	4,4×	4,3×	4×
8	0,7×	8,1×	10,1×	11,3×	8×
16	0,6×	16,8×	22,4×	23,4×	16×
32	0,8×	25,6×	43,0×	47,0×	32×
64	0,5×	27,2×	52,0×	78,3×	64×
128	0,2×	24,9×	52,3×	106,9×	128×

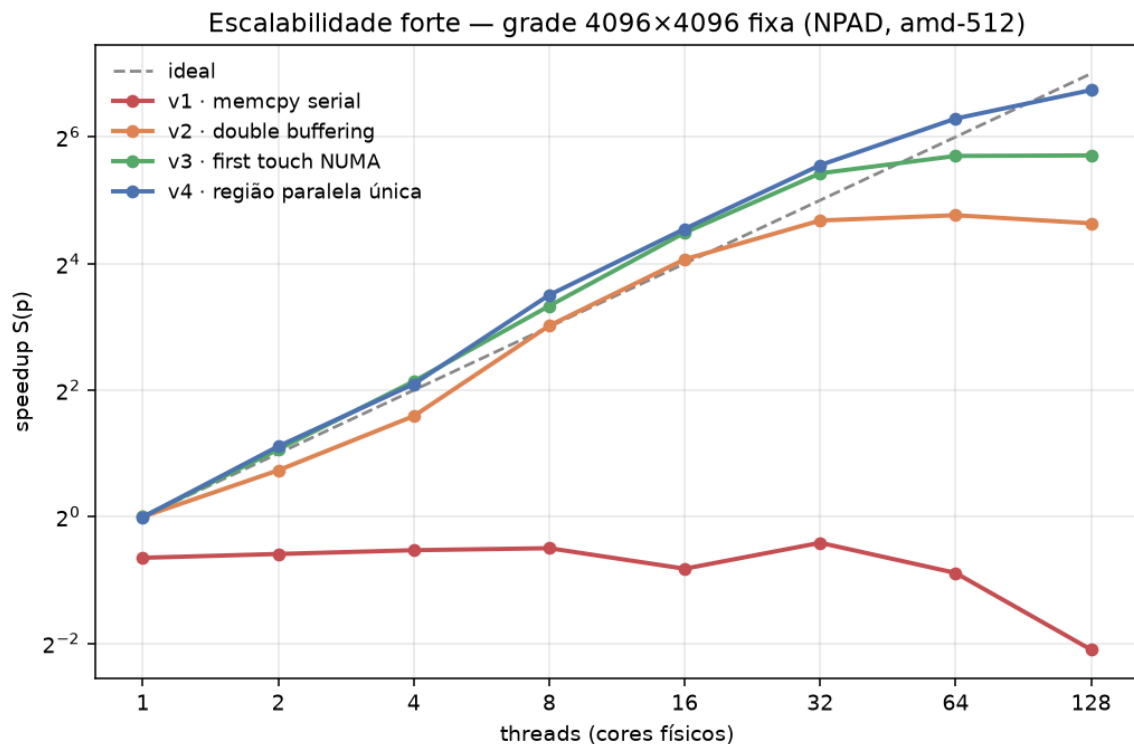


Figura 1 — Speedup com problema fixo. A v1 nunca supera 1×; cada versão seguinte desloca o joelho da curva para a direita.

6. Escalabilidade Fraca

Trabalho por thread constante: 2048^2 células por thread, ou seja $n = 2048 \cdot \sqrt{p}$. 50 passos, mediana de 3 execuções. O ideal é tempo constante, isto é, eficiência de 100%.

Threads	Grade	v1	v2	v3	v4	MLUPS (v4)
1	2048^2	100%	100%	100%	100%	1709
2	2896^2	36%	76%	91%	87%	2979
4	4096^2	24%	60%	81%	93%	6352
8	5793^2	12%	41%	69%	76%	10460
16	8192^2	5%	24%	63%	65%	17689
32	11585^2	4%	14%	29%	30%	16437
64	16384^2	2%	6%	12%	12%	13557
128	23170^2	1%	6%	6%	6%	13294

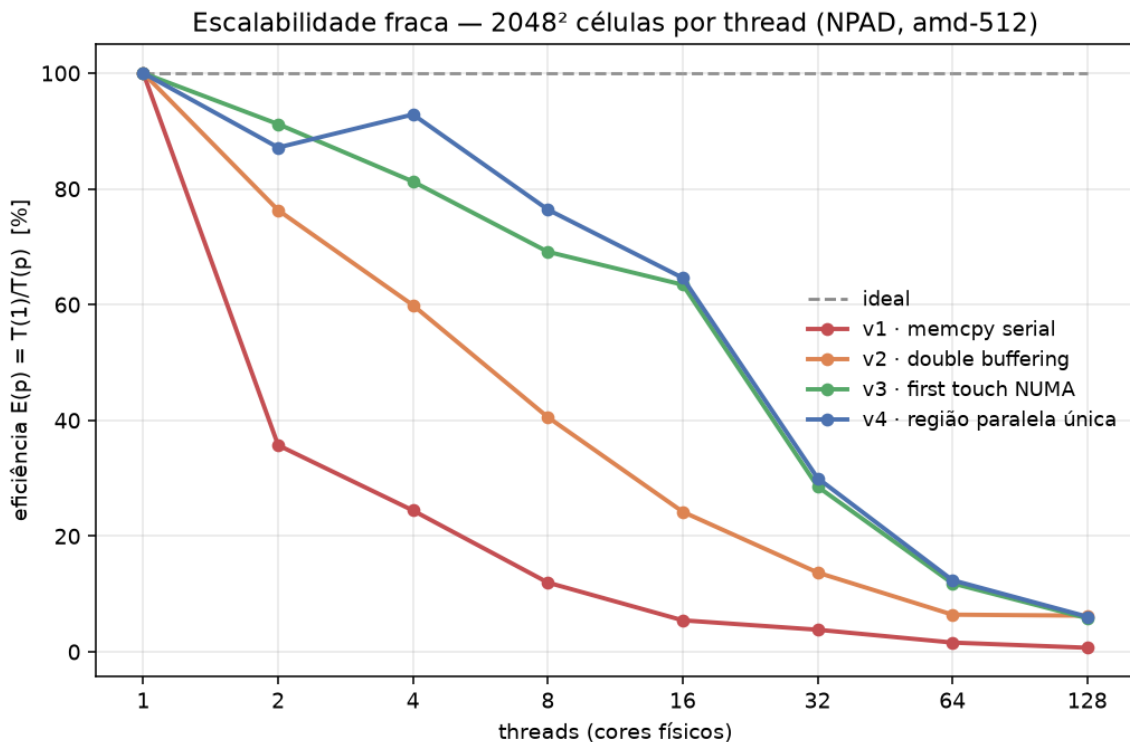


Figura 2 — Eficiência fraca. A queda comum a todas as versões a partir de 32 threads é o teto de banda de memória do nó, não um defeito do código.

7. Análise dos Gargalos

7.1 Trecho serial — Amdahl (v1 → v2)

A v1 calcula o grid novo em paralelo e copia tudo de volta com memcpy. Essa cópia move a mesma quantidade de bytes que o próprio stencil: cerca de metade do trabalho ficou serial, e por Amdahl o speedup satura em 2×. A medição é pior que a previsão — a v1 **nunca ultrapassa 0,8×** e com 128 threads chega a 5,0 s contra 1,17 s da sequencial. Amdahl supõe overhead zero; aqui, abrir e fechar um time de 128 threads a cada passo custa mais do que o paralelismo rende. Um trecho serial não apenas limita o ganho: em escala suficiente, ele o inverte. A correção não otimiza a cópia, elimina-a — trocam-se dois ponteiros.

7.2 Localidade NUMA — first touch (v2 → v3)

A v2 escala até 16 threads e então empaca. No Linux a página física é escolhida no primeiro acesso e nasce no domínio NUMA da thread que a tocou; alocar e zerar em serial coloca os 268 MB em um dos 8 domínios do nó. Quando o time se espalha pelos dois sockets, 7/8 dos acessos viram tráfego remoto e um único controlador de memória vira o gargalo da máquina inteira. A v3 não muda uma linha do stencil: apenas inicializa em paralelo com a mesma partição `static` do laço de cálculo, de modo que a thread que atualiza a linha *i* seja a que instancia suas páginas. Com 128 threads isso **dobra** o resultado: 24,9× → 52,3×.

7.3 Fork/Join por passo (v3 → v4)

A v3 satura em ~52×. A v4 abre uma única região paralela em torno de todo o laço do tempo, com cada thread deduzindo os buffers pela paridade do passo e a barreira implícita do `omp for` como único ponto de sincronização. A diferença de tempo por passo entre v3 e v4 isola o custo do fork/join:

Threads	16	32	64	128
$(T_{v3} - T_{v4})$ por passo	21 μs	23 μs	76 μs	114 μs

O custo **cresce com o número de threads**, como se espera de uma operação que acorda e recolhe p threads. Com 128 threads o passo da v3 leva 224 μ s e o da v4, 110 μ s: metade do tempo da v3 era fork/join, não cálculo. Daí o salto de 52,3 $\times$ para 106,9 $\times$. É o gargalo mais insidioso dos três, porque seu peso é absoluto por passo e não proporcional ao trabalho — invisível num grid grande, dominante num pequeno.

7.4 Eficiência acima de 100%

As tabelas mostram eficiência de até 147%, o que parece impossível. A explicação não é mérito do paralelismo: o conjunto de trabalho da escalabilidade forte são 268 MB, e enquanto uma thread sozinha enxerga os 32 MB de L3 do seu CCD, 128 threads espalhadas enxergam os 512 MB agregados do nó — o problema inteiro passa a caber em cache. O speedup superlinear mede a transição de DRAM para cache. Esse é o enunciado em estado puro: a v4 tem **desempenho** excelente nesse ponto, mas a **eficiência** está contaminada por uma mudança de regime de memória que não se repetiria num problema maior.

7.5 O teto restante — banda de memória

A escalabilidade fraca revela o limite real. A vazão cresce quase linearmente até 16 threads e então estabiliza em ~13300 MLUPS, aproximadamente 210 GB/s — cerca de metade do pico teórico do nó, valor típico para acesso de streaming. Um único núcleo já entrega ~1700 MLUPS. **A razão é 8 $\times$, não 128 $\times$.** Num kernel memory-bound o recurso que escala é a banda de memória, e ela cresce muito mais devagar que a contagem de núcleos. Nenhuma otimização de código atravessa esse teto; só mudaria o algoritmo, com temporal blocking.

8. Avaliação com o PaScaI Suite

O experimento foi repetido com o **PaScaI Analyzer** (LAPPS/IMD-UFRN), que varre o produto cartesiano de configurações e grava o JSON consumido pelo **PaScaI Viewer**. O binário instrumentado usa o mesmo núcleo numérico, com duas regiões marcadas: região 1 (inicialização) e região 2 (cálculo).

Os oito tamanhos foram casados às contagens de núcleos ($n = 2048 \cdot \sqrt{p}$), de modo que cada linha do mapa é uma escalabilidade forte e a diagonal é exatamente a escalabilidade fraca. As duas medições concordam: para $n = 2048$ com 1 núcleo, o sweep próprio mede 0,1763 s na v1 e o PaScaI, 0,1763 s. Dois instrumentos independentes chegando ao mesmo teto é o que permite atribuí-lo à máquina, e não ao método.

As Figuras 3 a 6 são os diagramas do próprio PaScaI Viewer, um por versão. Em todos, o eixo vertical vai de 2 a 128 núcleos (de cima para baixo) e o horizontal percorre os oito tamanhos, i1 a i8, em ordem crescente (2048 a 23170).

A progressão aparece como cor: o painel de eficiência vai de quase inteiramente amarelo na v1 — eficiência perto de zero em quase todo o espaço de configurações — a majoritariamente escuro na v3 e na v4. As duas últimas são quase indistinguíveis entre si, o que confirma o resultado da diagonal: o ganho da região paralela única aparece quando o passo de tempo é curto em relação ao custo de fork/join, e nesses tamanhos de problema a inicialização e o cálculo dominam. Nos mapas de v3 e v4 a região de alta eficiência fica à esquerda (i1–i4, os menores n), exatamente onde as tabelas registram os 135–145% de eficiência superlinear por residência em cache.

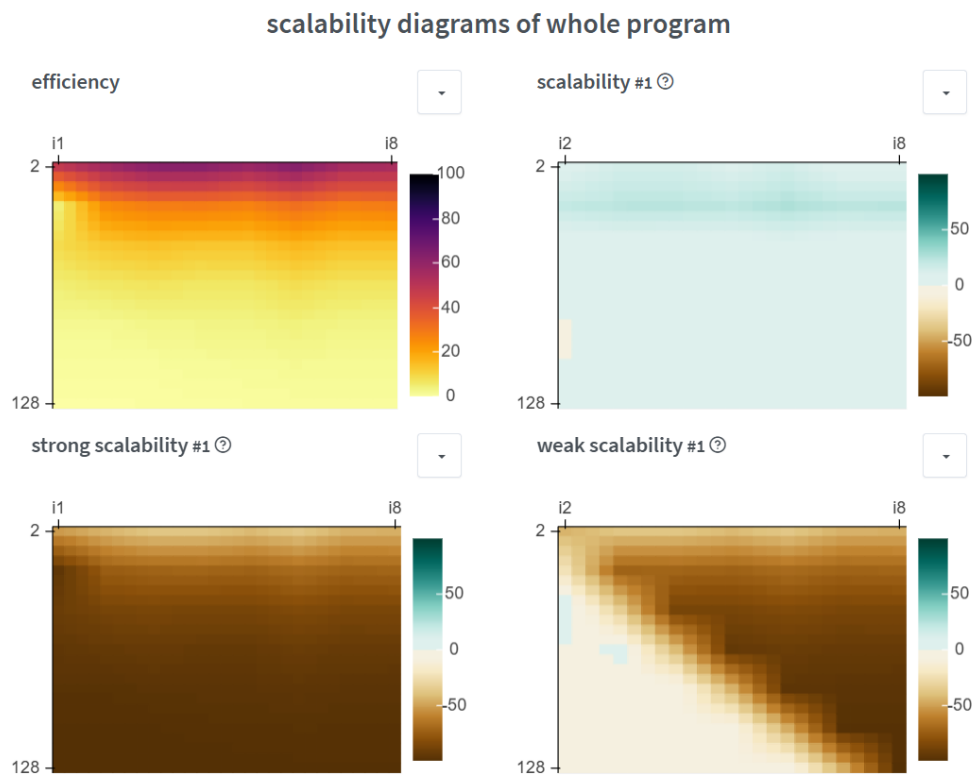


Figura 3 — v1 · memcpy serial. O painel de eficiência (superior esquerdo) é quase todo amarelo: fora da faixa de 2 núcleos, praticamente nada do recurso vira trabalho.

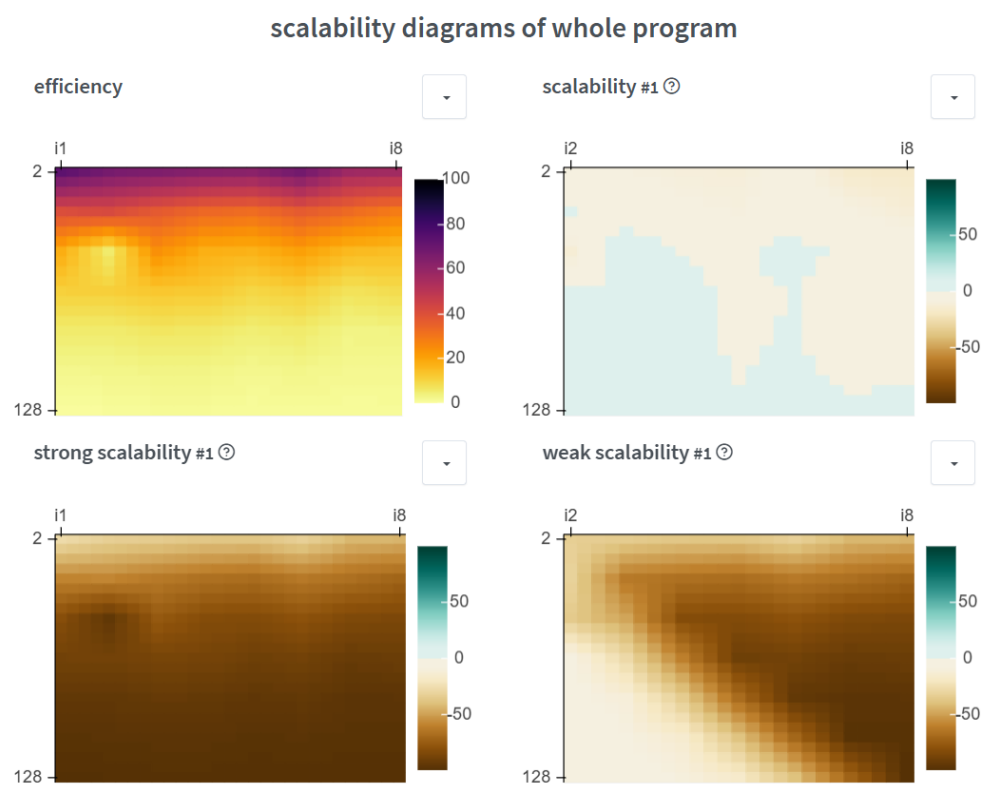


Figura 4 — v2 · double buffering. Removido o trecho serial, a faixa escura desce até cerca de 16 núcleos e então se dissolve.

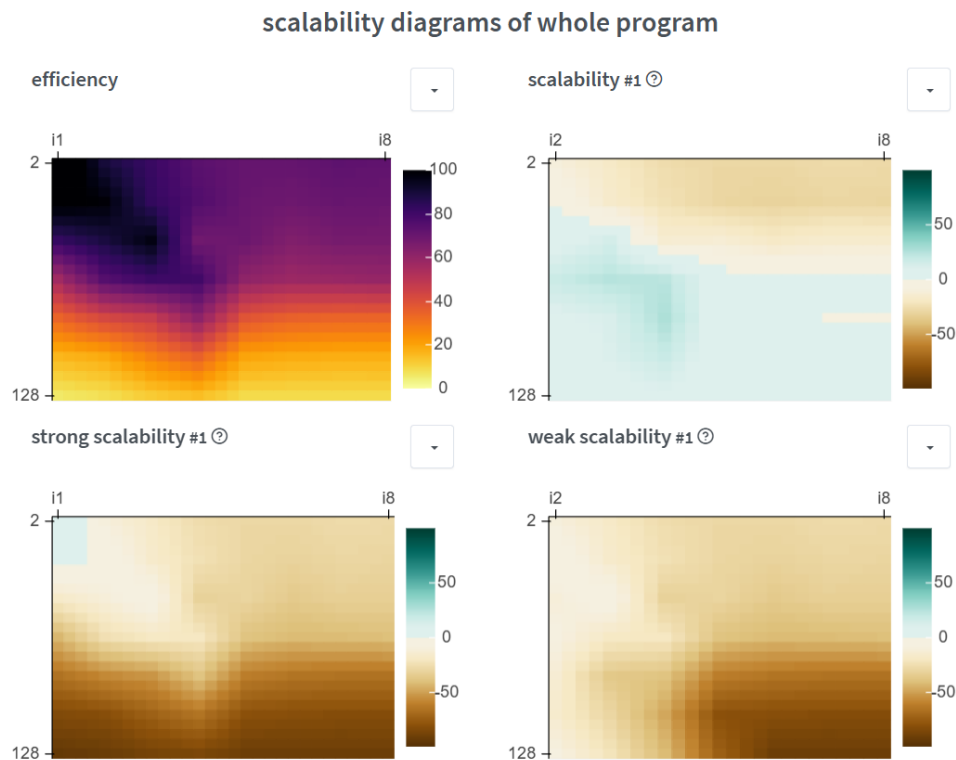


Figura 5 — v3 · first touch NUMA. A região escura se estende por boa parte do mapa; o degrau vertical por volta de i4–i5 marca onde o problema deixa de caber no cache agregado.

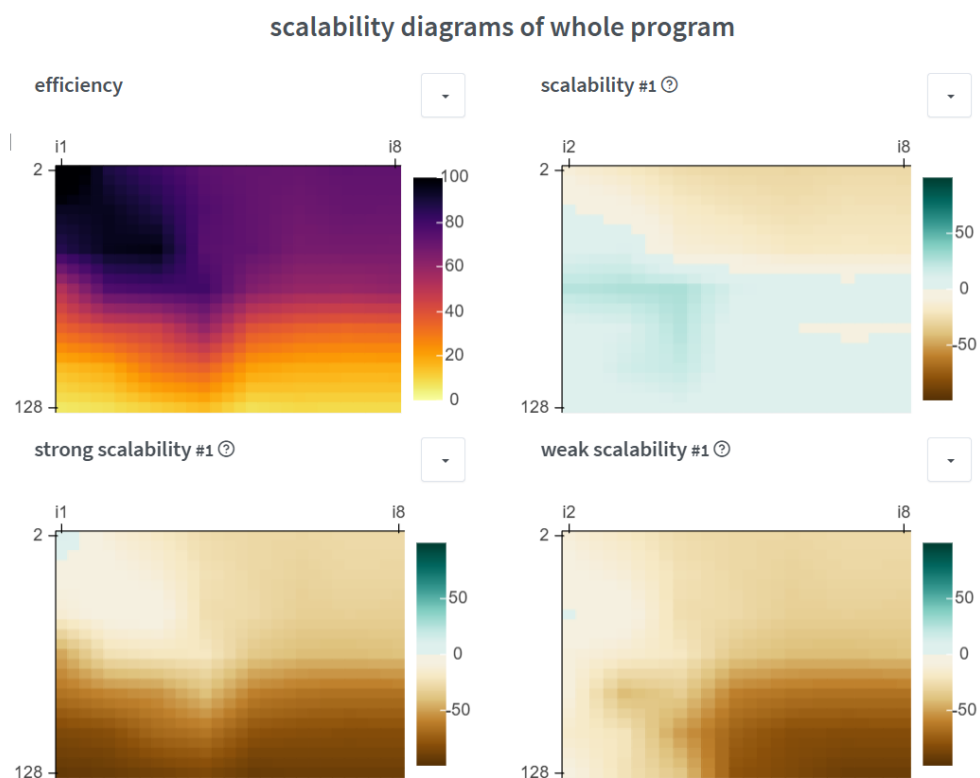


Figura 6 — v4 · região paralela única. Praticamente idêntica à v3 na visão do programa inteiro, onde o custo de fork/join é pequeno diante da inicialização.

8.1 O que só as regiões mostram

Separar inicialização de cálculo revelou o efeito mais direto de todo o trabalho. Tempo da região 1 com o maior problema ($n = 23170^2$):

Threads	v1	v2	v3	v4
1	13,73 s	13,72 s	14,14 s	14,15 s
8	13,72 s	13,72 s	1,82 s	1,83 s
32	13,73 s	13,73 s	0,48 s	0,48 s
128	13,72 s	13,73 s	0,19 s	0,19 s

A inicialização de v1 e v2 custa 13,7 s **em qualquer número de núcleos** — é serial por construção, e é exatamente ali que todas as páginas nascem em um único domínio NUMA. Em v3/v4 ela cai de 14,15 s para 0,19 s (74×). Isso responde a uma objeção que as tabelas de tempo de cálculo sozinhas não respondem: o ganho da v3 foi real ou apenas empurrado para fora da região medida? Foi real — a região 1 não engordou, encolheu 74×.

Vale notar que os diagramas do Viewer medem o *programa inteiro*, incluindo a região 1, e por isso são mais severos com v1 e v2 do que as tabelas das seções 5 e 6, restritas ao cálculo. Não é conflito, é escopo: o recorte do cálculo isola o efeito de cada otimização no laço; o do programa inteiro diz quanto disso o usuário final recebe.

9. Conclusão

Aplicando as definições do material à melhor versão (v4):

Pergunta	Resposta
É escalável?	Sim. A perda de eficiência causada pelo aumento de recursos é compensada pelo aumento do problema: de 1 para 128 threads a vazão sustentada sobe de 1,7 para 13,3 GLUPS.
É fracamente escalável?	Não — e nenhum código deste tipo poderia ser neste hardware. Exigiria que a banda de memória crescesse proporcionalmente aos núcleos; no EPYC 7713 ela cresce ~8× enquanto os núcleos crescem 128×. O limite é arquitetural, não de implementação.
É fortemente escalável?	Na faixa medida, sim: 106,9× com 128 threads (84% de eficiência) sem aumentar o problema. Com a ressalva da seção 7.4 — parte do resultado vem de a grade passar a caber no L3 agregado.

Progressão em uma linha:

Versão	Speedup em 128 threads	Gargalo removido
v1 · memcpy serial	0,2×	—
v2 · double buffering	24,9×	Trecho serial (Amdahl)
v3 · first touch NUMA	52,3×	Localidade de memória
v4 · região única	106,9×	Fork/join por passo
teto restante	—	Banda de memória (exigiria mudar o algoritmo)

A lição que atravessa as quatro versões é a do enunciado: **desempenho e escalabilidade são grandezas diferentes**. A v1 com 128 threads tem péssimo desempenho e péssima escalabilidade. A v4 na escalabilidade forte tem ótimo desempenho e ótima eficiência aparente — mas a escalabilidade fraca revela que boa parte disso era o problema ter encolhido para dentro do cache. Só medir os dois regimes distingue um código que escala de um código que apenas ficou rápido.

10. Código-Fonte

O arquivo `stencil.h` reproduzido a seguir concentra todo o desenvolvimento discutido neste relatório: os parâmetros físicos, as duas estratégias de alocação que distinguem a v2 da v3, o stencil de 5 pontos e as cinco funções `roda_v0 ... roda_v4`, onde se acompanha o `memcpy` desaparecer na v2 e a região paralela subir para fora do laço do tempo na v4.

Ficam de fora apenas os dois arquivos de infraestrutura de medição — `fluid_scale.c`, com os modos de validação e de benchmark, e `pascal_fluid.c`, com as marcações do PaScal —, que não contêm lógica do cálculo.

```
1  #ifndef STENCIL_H
2  #define STENCIL_H
3
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <string.h>
7  #include <math.h>
8  #include <omp.h>
9
10 #define NU 0.1          /* viscosidade cinemática  $\nu$  */
11 #define DX 1.0          /* espaçamento espacial */
12 #define DT 0.1          /* passo de tempo */
13 #define R (NU * DT / (DX * DX)) /*  $= 0.01 \leq 0.25 \rightarrow$  explícito estável */
14
15 #define AMPLITUDE 10.0
16 #define SIGMA0 5.0
17
18 /* — Alocação —————
19  *
20  * As duas rotinas devolvem a mesma coisa do ponto de vista do programa; o que
21  * muda é qual thread toca cada página primeiro*. No Linux a página física só
22  * é escolhida no primeiro acesso (first touch), e ela nasce no nó NUMA da
23  * thread que a tocou. Alocar e zerar em serial coloca o grid inteiro na
24  * memória de um socket só — as threads do outro socket passam a ler tudo pelo
25  * interconnect. Esse é o gargalo que separa v2 de v3.
26  */
27
28 static double *aloca_serial(size_t n) {
29     double *g = malloc(n * n * sizeof *g);
30     if (!g) { perror("malloc"); exit(EXIT_FAILURE); }
31     memset(g, 0, n * n * sizeof *g); /* toque serial: tudo num nó */
32     return g;
33 }
34
35 static double *aloca_first_touch(size_t n) {
36     double *g = malloc(n * n * sizeof *g);
37     if (!g) { perror("malloc"); exit(EXIT_FAILURE); }
38     /* Mesma partição estática do laço de cálculo: a thread que vai atualizar
39      a linha i é a que instancia as páginas da linha i. */
40     #pragma omp parallel for schedule(static)
```

Figura 7 — `stencil.h`, parte 1 de 4.

```

41     for (size_t i = 0; i < n; i++)
42         memset(&g[i * n], 0, n * sizeof *g);
43     return g;
44 }
45
46 /* — Condição inicial —————
47  * Gaussiana no centro. `paralelo` preserva o first touch de aloca_first_touch
48  * (mesma distribuição de linhas), então não desfaz a localidade NUMA.
49  */
50 static void inicia_perturbacao(double *u, size_t n, int paralelo) {
51     double ci = n / 2.0, cj = n / 2.0;
52     #pragma omp parallel for schedule(static) if (paralelo)
53     for (size_t i = 0; i < n; i++)
54         for (size_t j = 0; j < n; j++) {
55             if (i == 0 || j == 0 || i == n - 1 || j == n - 1) {
56                 u[i * n + j] = 0.0;          /* Dirichlet: borda fixa em 0 */
57                 continue;
58             }
59             double di = (double)i - ci, dj = (double)j - cj;
60             u[i * n + j] = AMPLITUDE * exp(-(di*di + dj*dj)
61                                     / (2.0 * SIGMA0 * SIGMA0));
62         }
63     }
64
65 /* — Stencil —————
66  * Uma linha interior do grid. Escrita contígua, cinco leituras vizinhas:
67  * ~6 flops para ~48 bytes de tráfego → o kernel é memory-bound, e é isso que
68  * define o teto de escalabilidade das versões já otimizadas.
69  */
70 static inline void linha(const double *u, double *un, size_t i, size_t n) {
71     const double *c = &u[i * n], *ci = &u[(i + 1) * n], *cd = &u[(i - 1) * n];
72     double *o = &un[i * n];
73     for (size_t j = 1; j < n - 1; j++)
74         o[j] = c[j] + R * (ci[j] + cd[j] + c[j+1] + c[j-1] - 4.0 * c[j]);
75 }
76
77 /* — v0: sequencial ————— */
78 static double roda_v0(double *u, double *un, size_t n, int nt) {
79     double t0 = omp_get_wtime();
80     for (int t = 0; t < nt; t++) {
81         for (size_t i = 1; i < n - 1; i++) linha(u, un, i, n);
82         double *tmp = u; u = un; un = tmp;
83     }
84     return omp_get_wtime() - t0;
85 }
86
87 /* — v1: paralela ingênua —————

```

Figura 8 — stencil.h, parte 2 de 4.

```

88  * O laço espacial é paralelo, mas o grid novo é copiado de volta sobre o
89  * antigo em serial. A cópia move exatamente a mesma quantidade de bytes que o
90  * próprio stencil – ou seja, ~50% do trabalho ficou serial. Por Amdahl o
91  * speedup satura perto de 2x, não importa quantos núcleos se acrescente.
92  */
93  static double roda_v1(double *u, double *un, size_t n, int nt) {
94      double t0 = omp_get_wtime();
95      for (int t = 0; t < nt; t++) {
96          #pragma omp parallel for schedule(static)
97          for (size_t i = 1; i < n - 1; i++) linha(u, un, i, n);
98          memcpy(u, un, n * n * sizeof *u);      /* trecho serial */
99      }
100     return omp_get_wtime() - t0;
101 }
102
103 /* — v2: double buffering —————
104  * A cópia vira uma troca de ponteiros: o trecho serial desaparece. Resta o
105  * fato de a memória ter sido tocada em serial (ver aloca_serial).
106  */
107  static double roda_v2(double *u, double *un, size_t n, int nt) {
108      double t0 = omp_get_wtime();
109      for (int t = 0; t < nt; t++) {
110          #pragma omp parallel for schedule(static)
111          for (size_t i = 1; i < n - 1; i++) linha(u, un, i, n);
112          double *tmp = u; u = un; un = tmp;
113      }
114      return omp_get_wtime() - t0;
115 }
116
117 /* — v3: v2 + first touch paralelo —————
118  * O laço é idêntico ao da v2. A diferença está fora da região medida: os
119  * grids vêm de aloca_first_touch, então cada thread lê e escreve páginas da
120  * memória do seu próprio socket. Em um nó de 2 sockets isso é a diferença
121  * entre usar a banda local e disputar o interconnect.
122  */
123  static double roda_v3(double *u, double *un, size_t n, int nt) {
124      return roda_v2(u, un, n, nt);
125 }
126
127 /* — v4: uma única região paralela —————
128  * v3 abre e fecha um time de threads a cada passo de tempo. Com 128 threads,
129  * esse fork/join custa dezenas de microssegundos – desprezível diante de um
130  * passo de 25 ms na escalabilidade forte, mas comparável ao passo quando o
131  * grid por thread é pequeno. Aqui o `omp parallel` envolve o laço do tempo e
132  * cada thread deduz os buffers a partir da paridade de t; a barreira implícita
133  * do `omp for` é o único ponto de sincronização por passo.

```

Figura 9 — stencil.h, parte 3 de 4.

```

134  */
135  static double roda_v4(double *u, double *un, size_t n, int nt) {
136      double t0 = omp_get_wtime();
137      #pragma omp parallel
138      {
139          for (int t = 0; t < nt; t++) {
140              const double *src = (t % 2 == 0) ? u : un;
141              double *dst = (t % 2 == 0) ? un : u;
142              #pragma omp for schedule(static)
143              for (size_t i = 1; i < n - 1; i++) linha(src, dst, i, n);
144              /* barreira implícita do `omp for` fecha o passo */
145          }
146      }
147      return omp_get_wtime() - t0;
148  }
149
150  /* — Infraestrutura de medição ————— */
151
152  typedef double (*roda_fn)(double *, double *, size_t, int);
153
154  static const struct { const char *nome; roda_fn fn; int first_touch; } VERSOES[] = {
155      { "v0-sequencial", roda_v0, 0 },
156      { "v1-memcpy", roda_v1, 0 },
157      { "v2-doublebuf", roda_v2, 0 },
158      { "v3-firsttouch", roda_v3, 1 },
159      { "v4-regiao-unica", roda_v4, 1 },
160  };
161  #define NVER ((int)(sizeof VERSOES / sizeof *VERSOES))
162
163  static double checksum(const double *u, size_t n) {
164      double s = 0.0; /* soma serial → determinística */
165      for (size_t k = 0; k < n * n; k++) s += u[k];
166      return s;
167  }
168
169  /* Depois de nt passos o campo final está ora em `u`, ora em `un`: toda versão
170     que alterna buffers termina em `un` para nt ímpar. A v1 é a exceção — ela
171     copia o resultado de volta para `u` a cada passo. */
172  static const double *resultado(int v, const double *u, const double *un, int nt) {
173      if (v == 1) return u;
174      return (nt % 2) ? un : u;
175  }
176
177  #endif /* STENCIL_H */
178

```

Figura 10 — stencil.h, parte 4 de 4.